SHOPSTOCK

Inventory Management System

Setup & Workflow Guide

Version	ShopStock v4.0
Stack	Python / Flask · MySQL · Gunicorn · Jinja2
Date	27 May 2026
Purpose	Installation, configuration, and workflow reference for administrators and cashiers

1. Project Overview

ShopStock is a role-aware, web-based inventory management system for small-to-medium retail shops. It provides real-time stock control, point-of-sale billing, supplier and customer management, and business reporting. Two roles exist: Admin (full access) and Cashier (POS-focused). Gunicorn is used in production to support multiple concurrent cashier sessions.

2. Prerequisites

Requirement	Version
Python	3.10 or higher
MySQL	8.0 or higher
pip	Latest (bundled with Python)
Gunicorn	Installed via pip — Linux/macOS only (use Flask dev server on Windows or WSL2)

3. Installation & Setup

Step 1 — Clone or copy the project

```
git clone <your-repo-url> shopstock

cd shopstock
```

Or unzip the project archive and enter the folder.

Step 2 — Create a virtual environment

```
python3 -m venv venv

source venv/bin/activate          # Linux / macOS

venv\Scripts\activate             # Windows
```

Step 3 — Install dependencies

```
pip install flask>=3.0.0 mysql-connector-python>=8.0.0 gunicorn
```

Or if a requirements.txt is present:

```
pip install -r requirements.txt
```

Step 4 — Configure the database connection

Open app.py and update the DB_CONFIG block near the top:

```
DB_CONFIG = {
```

```
    "host":     "localhost",
    "user":     "your_mysql_user",
    "password": "your_mysql_password",
    "database": "shopstock",
}
```

4. Database Setup

Step 1 — Create the database

```
CREATE DATABASE shopstock CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Step 2 — Create the tables

Run the schema SQL provided separately (schema.sql) inside the shopstock database, or paste the CREATE TABLE statements for: users, suppliers, products, customers, sales, sale_items, and activity_log. See the System Overview Report for full table definitions.

Step 3 — Create the first admin user

ShopStock passwords are stored as SHA-256 hashes. Generate one in Python:

```
import hashlib
pw = hashlib.sha256("your_password_here".encode()).hexdigest()
print(pw)
```

Then insert the admin user into MySQL:

```
INSERT INTO users (username, password_hash, role)
VALUES ('admin', '<paste_hash_here>', 'admin');
```

To add a cashier account:

```
INSERT INTO users (username, password_hash, role)
VALUES ('cashier1', '<paste_hash_here>', 'cashier');
```

5. Running the Application

Development — Flask built-in server

Suitable for local testing only. Single-threaded, not for production use.

```
python app.py
```

Access at: http://127.0.0.1:5000

Production — Gunicorn (Linux / macOS)

Gunicorn runs multiple worker processes so cashiers can log in concurrently without blocking each other.

```
gunicorn -w 4 -b 0.0.0.0:8000 app:app
```

Flag	Meaning
-w 4	4 worker processes (rule of thumb: 2 x CPU cores + 1)
-b 0.0.0.0:8000	Listen on all interfaces, port 8000
app:app	filename:flask_instance_name

Access at: http://<your-server-ip>:8000

Running as a background service (systemd)

Create /etc/systemd/system/shopstock.service with the following content:

```
[Unit]

Description=ShopStock Gunicorn Service

After=network.target

[Service]

User=www-data

WorkingDirectory=/path/to/shopstock

ExecStart=/path/to/shopstock/venv/bin/gunicorn -w 4 -b 0.0.0.0:8000 app:app

Restart=always



[Install]

WantedBy=multi-user.target
```

Then enable and start:

```
sudo systemctl daemon-reload

sudo systemctl enable shopstock

sudo systemctl start shopstock
```

```
sudo systemctl status shopstock
```

6. User Workflows

6.1 Admin Workflow

Logging In

- Open the app in a browser.
- Select the Admin tab on the login screen.
- Enter your username and password and click Sign In.

Adding a Product

- Go to Products in the sidebar.
- Click + Add Product.
- Fill in the name, category, price, and initial stock.
- Select an existing supplier or type a new one to create it inline.
- Click Save — the SKU is auto-generated (PRD001, PRD002…).

Restocking / Adjusting Stock

- On the Products screen, find the product row.
- Click Restock IN to add units or Restock OUT to remove.
- Enter the quantity and an optional note, then click Confirm.
- The activity log is updated immediately.

Viewing Sales & Refunding

- Go to Sales in the sidebar.
- Use the date / customer / status filters to find a sale.
- Click Refund on a completed sale to restore stock and mark it refunded.
- Optionally click Delete to permanently remove the refunded record.

Viewing Reports

- Go to Reports in the sidebar.
- Choose a report type: Daily, Monthly, Inventory, or Stock Health.
- Set the filters (date, supplier, stock status) and click Load.

6.2 Cashier Workflow

Logging In

- Open the app in a browser.
- Select the Cashier tab on the login screen.
- Enter your username and password and click Sign In as Cashier.

Processing a Sale

- Go to Billing Counter in the sidebar — this is the main POS screen.
- Browse or search for products in the grid. Use category filters to narrow down.

- Click a product card to add it to the cart (right panel). Adjust quantity as needed.
- To attach a customer, type their name or phone in the Customer field. If they exist, select from the dropdown; if new, just type their name — they will be auto-created on sale completion.
- Add an optional note if needed, then click Complete Sale.
- Stock is decremented in real time and an invoice is generated automatically.
- Click Print / Save PDF on the invoice page to give the customer a copy.

Viewing My Sales

- Go to My Sales in the sidebar.
- Filter by Today, This Week, or This Month.
- Click Invoice on any row to reopen the printable invoice.

Alerting the Admin (Low Stock)

- From the Cashier Dashboard, click Alert Admin (top right).
- Either click a Quick Fill button for a known low-stock item, or fill in the SKU, product name, current stock, and message manually.
- Click Send Alert — the admin will see a popup notification on their next login.

7. Route Reference

Method	Route	Role	Description
GET/POST	/login	Public	Login page
GET	/logout	Any	Log out and clear session
GET	/dashboard	Admin	Admin dashboard
GET	/cashier/dashboard	Cashier	Cashier dashboard
GET	/products	Admin	Product catalogue
POST	/products/add	Admin	Add a new product
POST	/products/restock	Admin	Restock IN or OUT
POST	/products/price	Admin	Update product price
POST	/products/delete	Admin	Delete a product
GET	/billing	Cashier	POS billing screen
POST	/sales/add	Cashier	Submit a cart / complete a sale
GET	/sales	Admin	View all sales
GET	/sales/invoice/<id>	Admin/Cashier	View / print invoice
POST	/sales/refund/<id>	Admin	Refund a sale
POST	/sales/delete/<id>	Admin	Delete a refunded sale
GET	/cashier/sales	Cashier	Cashier's own sales
GET	/suppliers	Admin	Supplier directory
GET	/customers	Admin/Cashier	Customer list
GET	/activity_log	Admin	Full activity log
GET	/reports	Admin	Reports module
POST	/cashier/alert/send	Cashier	Send low-stock alert (JSON)

8. Project Structure

```
shopstock/
|
+-- app.py                      # Main Flask application — all routes & DB
logic
|
+-- templates/                  # Jinja2 HTML templates
|   +-- base.html               # Shared layout, sidebar, navigation
|   +-- login.html              # Login page (Admin / Cashier tabs)
|   +-- dashboard.html          # Admin dashboard
|   +-- cashier_Dashboard.html # Cashier dashboard
|   +-- products.html           # Product management
|   +-- billing.html            # POS billing screen
|   +-- sales.html              # Admin sales view
|   +-- Cashier_my_sales.html   # Cashier's own sales
|   +-- Invoice.html            # Printable invoice
|   +-- suppliers.html          # Supplier directory
|   +-- activity_log.html       # Audit log
|   +-- reports.html            # Reports module
|
+-- requirements.txt            # Python dependencies
+-- README.docx                # This document
```

9. Troubleshooting

Problem	Solution
Cannot connect to MySQL	Check DB_CONFIG in app.py. Confirm MySQL is running and the user has privileges on the shopstock database.
'Table doesn't exist' error	Re-run the schema SQL from Section 4. All 7 tables must be created before starting the app.
Login fails with correct password	The password is stored as a SHA-256 hash. Make sure you inserted the hash (not the plain-text password) into the users table.
Gunicorn not found	Make sure your virtual environment is active and gunicorn was installed inside it: pip install gunicorn.
Port already in use	Change the port: gunicorn -w 4 -b 0.0.0.0:5001 app:app and access at port 5001.
Stock goes negative	This should not happen — the app validates stock with a FOR UPDATE lock before committing. If seen, check that no one is writing directly to the database outside the application.

Internal Documentation